A REPORT

ON

PART PROCUREMENT INTELLIGENCE SYSTEM

BY

POLAMARASETTY BHASHINI

AT

HINDUSTAN AERONAUTICS LIMITED, HYDERABAD

An Internship Program

DECLARATION

I hereby declare that the Internship report submitted along with the Internship entitled “PART PROCUREMENT INTELLIGENCE SYSTEM” submitted in fulfillment of the internship program for the degree in Bachelor of Technology in Computer Science Engineering of GITAM University Bengaluru, is a Bonafide record of original project work carried out by me at “HINDUSTAN AERONAUTICS LIMITED (HAL), Hyderabad” under the supervision of Mr. J. VEERANJANEYULU, Chief Manager (IT) (Guide) and that no part of this report has been directly copied from any students reports or taken from any other source, without providing due reference.

ACKNOWLEDGEMENT

I am extremely grateful for the support and encouragement we received throughout my internship at Hindustan Aeronautics Limited, and I would like to seize this opportunity to express my sincere appreciation to everyone who contributed to my learning and development during this period.

First and foremost, I would like to thank my internship guide Shri J. VEERANJANEYULU, Chief Manager, IT Department, Hindustan Aeronautics Limited, Avionics Division, Hyderabad for his stimulating guidance. I shall always cherish my association with his encouragement, and valuable suggestions throughout the progress of this work. I consider it a great privilege to work under his guidance and constant support. I would also like to thank him for helping me to complete the Internship by taking frequent reviews throughout the progress of this internship. I thank everyone who helped me directly and indirectly throughout this Internship.

Moreover, I would like to express gratitude to all our team members for their willingness to aid me and for generously sharing their expertise.

Lastly, I would like to acknowledge the entire staff at Hindustan Aeronautics Limited company for fostering a conductive and nurturing environment, where I could explore and learn with enthusiasm.

In conclusion, this internship has been a rewarding experience, and I am grateful to each and every individual who played a part in making it a memorable and valuable journey.

1. INTRODUCTION

1.1 OVERVIEW

In heavy-manufacturing ecosystems like **Hindustan Aeronautics Limited (HAL)**, operational efficiency is closely tied to the precision of the procurement cycle. The production of advanced avionics and aircraft configurations requires thousands of specialized components, assembly parts, and raw materials, often sourced globally.

A single delayed component can stall downstream assembly lines, alter manufacturing schedules, and create costly resource bottlenecks. Historically, tracking these materials relied on standard historical averages or rigid deterministic buffers. While simple, these traditional methods fail to capture modern logistical complexities, currency fluctuations, or administrative friction.

To shift from a reactive to a proactive planning model, this project introduces an intelligence-driven solution: the **Supply Chain Lead Time Prediction & Analytics Engine**. This system acts as an automated enterprise utility that processes historical procurement histories, targets exact processing intervals, and applies machine learning algorithms to forecast critical material arrival timelines.

1.2 PURPOSE AND OBJECTIVES

The primary goal of this system is to replace arbitrary delivery windows with data-driven, mathematically sound arrival predictions. Rather than treating the procurement cycle as a single opaque block of time, the engine decomposes the lifecycle into distinct structural intervals.

Specifically, the system isolates two key operational gaps:

1. Gap 1 (Req ➔ PO): Administrative Turnaround Time

This interval spans from the initialization of an official material requirement to the formal release of a purchase order.

Gap 1 = Purchase Order (PO) Date - Requisition Date

2. Gap 2 (PO ➔ Gate): Supplier & Logistical Transit Time

This interval captures the time from purchase order confirmation to the arrival and processing of physical inventory at the company's receiving facility.

Gap 2 = Gate Entry / Arrival Date - Purchase Order (PO) Date

By training separate regression models on these distinct segments, the system helps management isolate exactly where delays occur. High values in **Gap 1** flag internal administrative bottlenecks or approval delays, whereas spikes in **Gap 2** signal external supplier constraints or shipping disruptions.

The unified output of these models yields the total predicted **Lead Time (Gap 3)**, mapping out exact milestones that feed directly into production planning dashboards.

Lead Time (Gap 3) = Gap 1 + Gap 2

2. TECHNOLOGY STACK & SYSTEM ENVIRONMENT

To ensure structural stability, enterprise security, and seamless deployment within the local server framework at Hindustan Aeronautics Limited (HAL), the machine learning infrastructure was built using a highly stable, decoupled open-source technology stack.

The environment is designed to handle high-cardinality transactional features, process data with minimal latency, and interface cleanly with enterprise Business Intelligence (BI) visualization layers without requiring live, high-overhead database connections during the initial rollout.

2.1 EXECUTION ENVIRONMENT & RUNTIME

- **Host Distribution Suite: Anaconda (Version 1.2.2 Platform Layout)**

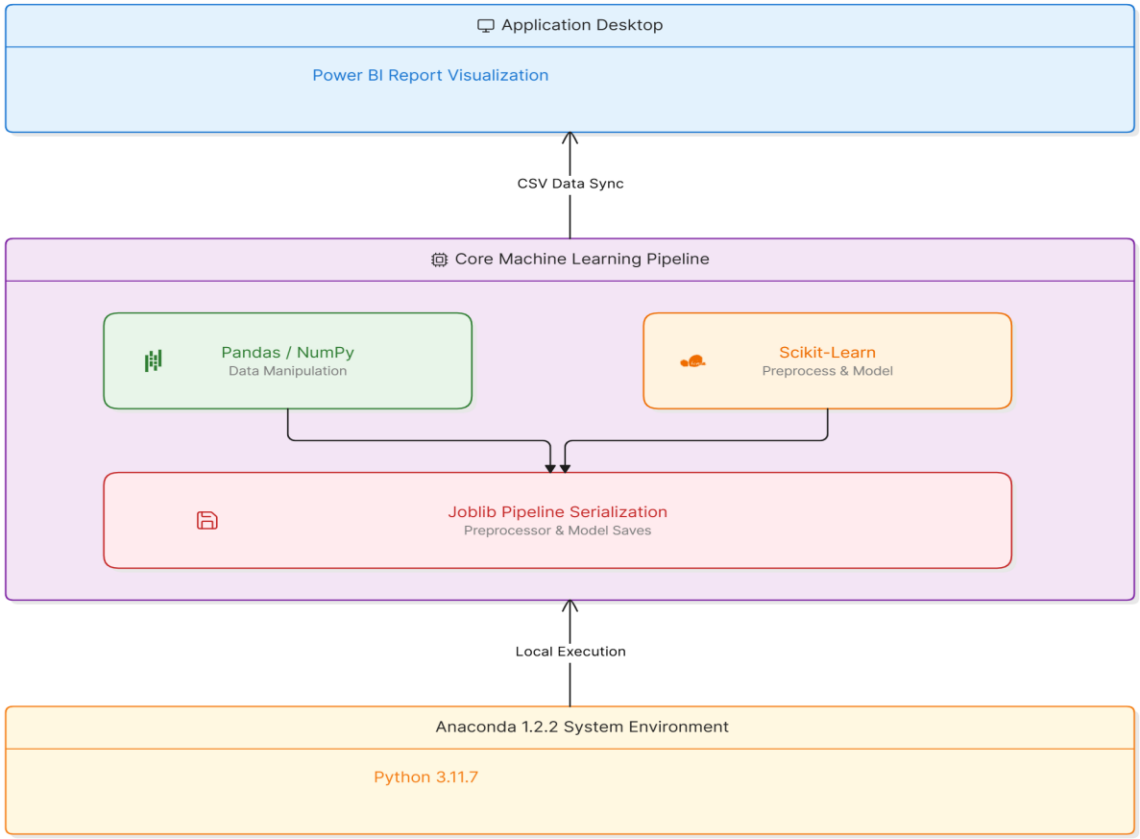
  Role in Project: Anaconda was selected to provide an enterprise-ready, isolated Python workspace. This local containment prevents software dependency updates on the host operating system from breaking production pipelines, ensuring consistent runs across multi-user environments.

- **Programming Language Engine: Python (Version 3.11.7)**

  Role in Project: Python 3.11.7 provides the core engine for this project. This runtime release balances modern optimization features, such as enhanced CPU execution speeds and streamlined memory footprinting, with broad compatibility across foundational mathematical and machine learning libraries.

2.2 SYSTEM LIBS & MACHINE LEARNING FRAMEWORKS

The software relies on a core group of libraries to manage data pipelines, train regression models, and handle file serialization:



2.2.1 Data Ingestion and Matrix Engineering

- **Pandas Engine (Optimized Core):** Used to load raw historical text files, handle date casting across varied file exports, and group rows together to support target lookup functions.
- **NumPy Array Foundations:** Converts tabular spreadsheet data into high-performance numeric vectors. This library handles the final horizontal concatenation step (np.hstack), combining base one-hot encoded characteristics with custom component-level target metrics instantly.

2.2.2 Preprocessing & Transformer Pipeline

- **Scikit-Learn (Version 1.2.2 Architecture):** The primary machine learning framework used throughout the development cycle.
- **ColumnTransformer & Pipeline**: Combines missing-value imputation, variable tracking, and calendar transformations into a single, cohesive unit. This structure guarantees that data processing rules are applied identically during both model training and real-time production inference.
- **SimpleImputer**: Handles sparse data anomalies by replacing missing values using target median and constant fill strategies.
- **OneHotEncoder**: Converts text labels (such as countries and currency types) into binary vectors, using an explicit fallback configuration (handle_unknown='ignore') to maintain pipeline stability when processing unexpected inputs.

2.2.3 Core Modeling Algorithms

- **HistGradientBoostingRegressor:** An advanced tree-boosting regressor tailored for large datasets. By binning continuous variables into discrete integer intervals, it avoids the need to sort floating-point values at every tree split, accelerating training across thousands of high-cardinality items.

2.2.4 Object Persistence and Serialization

- **Joblib Engine:** Used to save trained models and preprocessors to disk as serialized binaries (.joblib). This allows the inference system to load and execute pre-trained pipelines instantly, separating the prediction layer from the high-overhead training logic.

2.3 ENTERPRISE VISUALIZATION AND REPORTING LAYER

- **Visual Delivery Platform: Power BI Desktop**

Role in Project: Power BI handles the user-facing reporting layer, turning complex model outputs into clean, interactive dashboards. The application consumes a clean prediction export (powerbi_predictions_feed.csv), decoupling heavy back-end machine learning operations from the end-user reporting interface. This approach provides procurement managers with real-time access to predicted timelines, lead-time distribution maps, and supplier bottleneck flags without requiring local Python execution environments.

3. ARCHITECTURAL LAYOUT AND MODULE DESCRIPTIONS

To ensure the long-term maintainability, reliability, and auditability of the forecasting engine within the HAL IT environment, the software architecture is structured as a fully decoupled, production-grade machine learning system.

The system isolates individual software responsibilities, such as configuration definitions, variable transformers, training execution loops, runtime verification checks, and end-user inference engines, into separate, specialized scripts.

3.1 SYSTEM FLOW ARCHITECTURE

The diagram below illustrates how procurement information moves through the decoupled architecture: from raw source files, through multi-channel preprocessing steps and concurrent gradient boosting estimators, and finally into the target Power BI reporting environment.

3.2 DETAILED FILE-LEVEL COMPONENT BREAKDOWN

The deployment package is organized into the following file layout within the root folder (procurement_ml_v3/):

3.2.1 src/config.py (Enterprise Definition Control)

- **Functional Description:** This module serves as the central directory for all system properties, file routes, and operational metadata.

- **Core Logic & Responsibilities:**
  1. Establishes absolute file paths for raw assets (data/) and model checkpoints (models/) using path evaluations relative to the codebase root.
  2. Declares structural feature classifications, including standard categories (COUNTRY, CURRENCY_CODE), high-cardinality values (PART_NO), timelines (REQUISITION_DATE), and continuous financial fields (CURRENCY_RATE).
  3. Locks in repeatable experiment variables, specifying a validation holdout split of 0.2 alongside a set random seed value of 42.

4. Names global system keys for the internal targeted timelines, gap1 and gap2.

3.2.2 src/data_preprocessing.py (Multi-Channel Pipeline Engine)

- **Functional Description:** Constructs and returns a structured ColumnTransformer object, establishing parallel data manipulation pipelines.
- **Core Logic & Responsibilities:**
- **Numerical Path:** Directs continuous numerical entries into an imputation process that uses a median value replacement strategy to handle missing entries.
- **Categorical Path:** Automatically isolates structural labels, handles missing fields with a constant 'UNKNOWN' assignment, and applies a non-sparse OneHotEncoder that ignores unmapped parameters during active inference workflows.
- **Temporal Path:** Passes datetime sequences directly to custom calendar extraction utilities.
- DROPS any unused supplementary column headers by applying a strict remainder='drop' rule. This ensures that the downstream matrix shape remains consistent and predictable.

3.2.3 src/feature_engineering.py (Temporal Component Extractor)

- **Functional Description:** Implements the custom ProcurementDateTransformer class, extending the standard scikit-learn pipeline ecosystem.
- **Core Logic & Responsibilities:**
- Explicitly parses input columns using localized European timestamp formats (dayfirst=True) and handles corrupted records cleanly using errors='coerce'.
- Implements an inference fallback routine: if any timestamps are missing during live production prediction queries, the system automatically patches those fields with the current server time (pd.Timestamp.now()).
- Decomposes individual raw dates into three separate numerical inputs: Month, Day of the week, and Year-quarter. It then drops the original base string index to prevent array parsing issues downstream.

3.2.4 src/model.py (Estimator Hyperparameter Blueprint)

- **Functional Description:** Defines the structural hyperparameters for the core machine learning models.
- **Core Logic & Responsibilities:**
- Initializes and returns instances of the HistGradientBoostingRegressor.
- Hardcodes standard training parameters optimized for medium-scale enterprise datasets: max_iter=150, learning_rate=0.08, max_depth=6, and a minimum node constraint of min_samples_leaf=20.

3.2.5 src/predict.py (Production Inference & Search Query Engine)

- **Functional Description:** The system's operational core. It processes input payloads, reconciles variable array lengths, evaluates historical information slices, and generates structured outputs.
- **Core Logic & Responsibilities:**
- Loads serialized pipeline weights, target encoding lookups, and historical file repositories on startup.
- Ensures consistent string lengths by applying an automatic zero-padding rule (str.zfill(8)) across all component part IDs.
- Dynamically extracts high-cardinality historical target encodings, applying default global fallback metrics if an item has not been seen before.
- Corrects potential structural vector length variances by enforcing an absolute length limit of 8402 features across inference arrays.
- Runs parallel prediction models, limits calculated values to logical bounds (preventing negative values), applies mathematical identity checks, and formats dates to standard layouts (%d-%m-%Y) for clean synchronization with Power BI.

3.2.6 src/evaluate.py (Validation Verification Tool)

- **Functional Description:** An execution module that verifies the performance, accuracy, and structural separation of saved pipeline files.

- **Core Logic & Responsibilities:**
- Confirms object serialization independence by attempting to load saved model structures without referencing any underlying training components.
- Re-reads historical records and calculates real-time evaluation indices, such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared scores (R^2) across unseen verification arrays.

3.2.7 run_e2e_test.py (Root-Level Automation Pipeline)

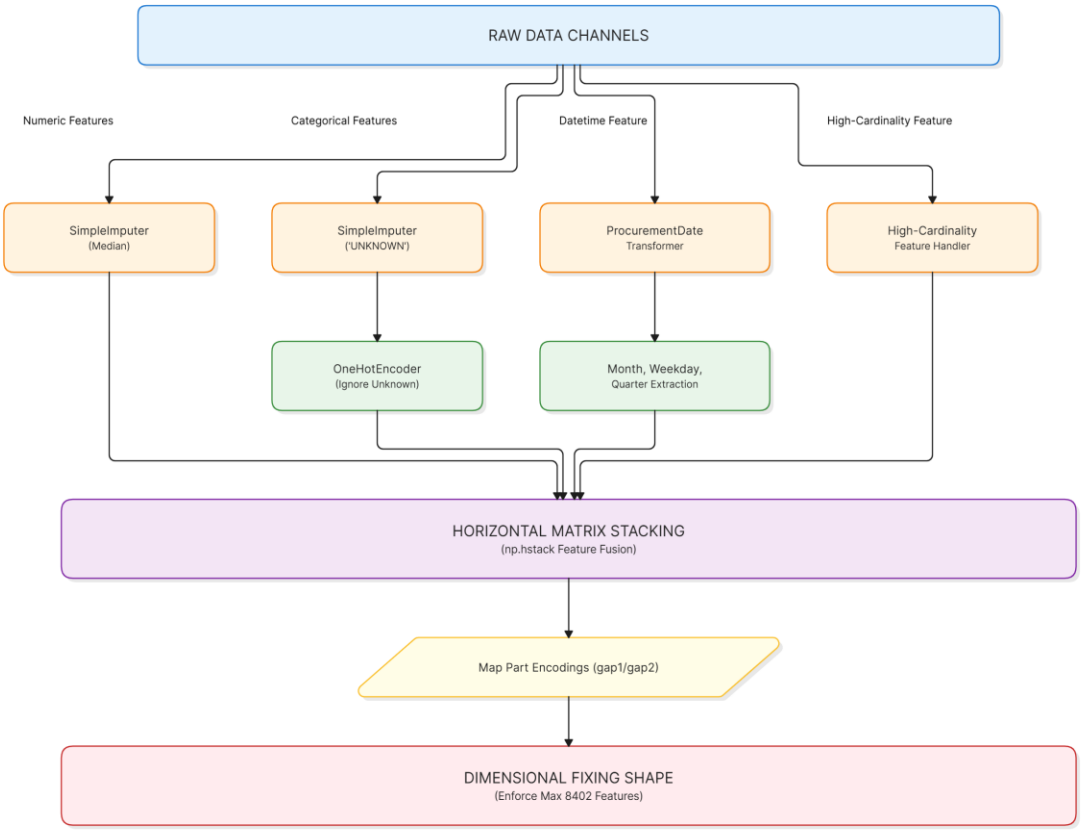
- **Functional Description:** A system-wide execution script that resides in the root directory to confirm overall program stability.
- **Core Logic & Responsibilities:**
- Confirms that necessary runtime sub-folders (data/, models/) exist on the host server.
- Validates the presence of base historical records, alerting users if source files are missing.
- Triggers training processes, runs localized validation data payloads, and verifies that the system's underlying mathematical identity equations remain unbroken.

4. FEATURE ENGINEERING AND PREPROCESSING PIPELINE

To handle the variety of structured fields in enterprise procurement records, the system processes raw transaction inputs through decoupled data-transformation streams. This ensures that the downstream gradient boosting models receive highly optimized numeric vectors, while preventing data leakage between the training and testing sets.

4.1 STRATIFIED BASE CHARACTERISTIC TRANSFORMATIONS

The system passes input features through an automated, multi-channel ColumnTransformer layout. This architecture isolates columns based on their operational profiles and routes them through independent processing pathways:



4.1.1 Continuous Numerical Channel

- **Target Vector:** CURRENCY_RATE
- **Transformation Strategy:** Implemented via a scikit-learn Pipeline utilizing SimpleImputer(strategy='median').
- **Design Rationale:** Procurement histories can have missing data due to unrecorded currency exchanges. Using a median replacement strategy protects the model from extreme spikes or data anomalies, ensuring a stable baseline value for local currency conversions.

4.1.2 Low-Cardinality Categorical Channel

- **Target Vectors:** COUNTRY, CURRENCY_CODE
- **Transformation Strategy:** Processed sequentially using an explicit missing-value assignment (SimpleImputer(strategy='constant', fill_value='UNKNOWN')) followed by an isolated one-hot vector transformation (OneHotEncoder(handle_unknown='ignore', sparse_output=False)).
- **Design Rationale:** In manufacturing environments, incoming parts may occasionally come from new suppliers or countries. The handle_unknown='ignore' property prevents runtime errors by encoding novel categories as a vector of all zeros, maintaining systemic stability during active inference operations.

4.2 CYCLICAL DATETIME EXTRACTION & PRODUCTION FALLBACKS

Temporal features like REQUISITION_DATE cannot be used directly by tree-based models as raw text strings. The engine uses a custom pipeline transformer, ProcurementDateTransformer, to extract cyclical calendar metrics from these fields.

4.2.1 Operational Pipeline Resilience

During live inference queries, incoming data payloads may contain missing or corrupt date entries. To protect against system crashes, the custom transformer checks data integrity using localized datetime methods:

```
Python
# Production fallback strategy for missing timestamps during inference
if dates.isnull().any():
    dates = dates.fillna(pd.Timestamp.now())
```

4.2.2 Feature Dimensional Decomposition

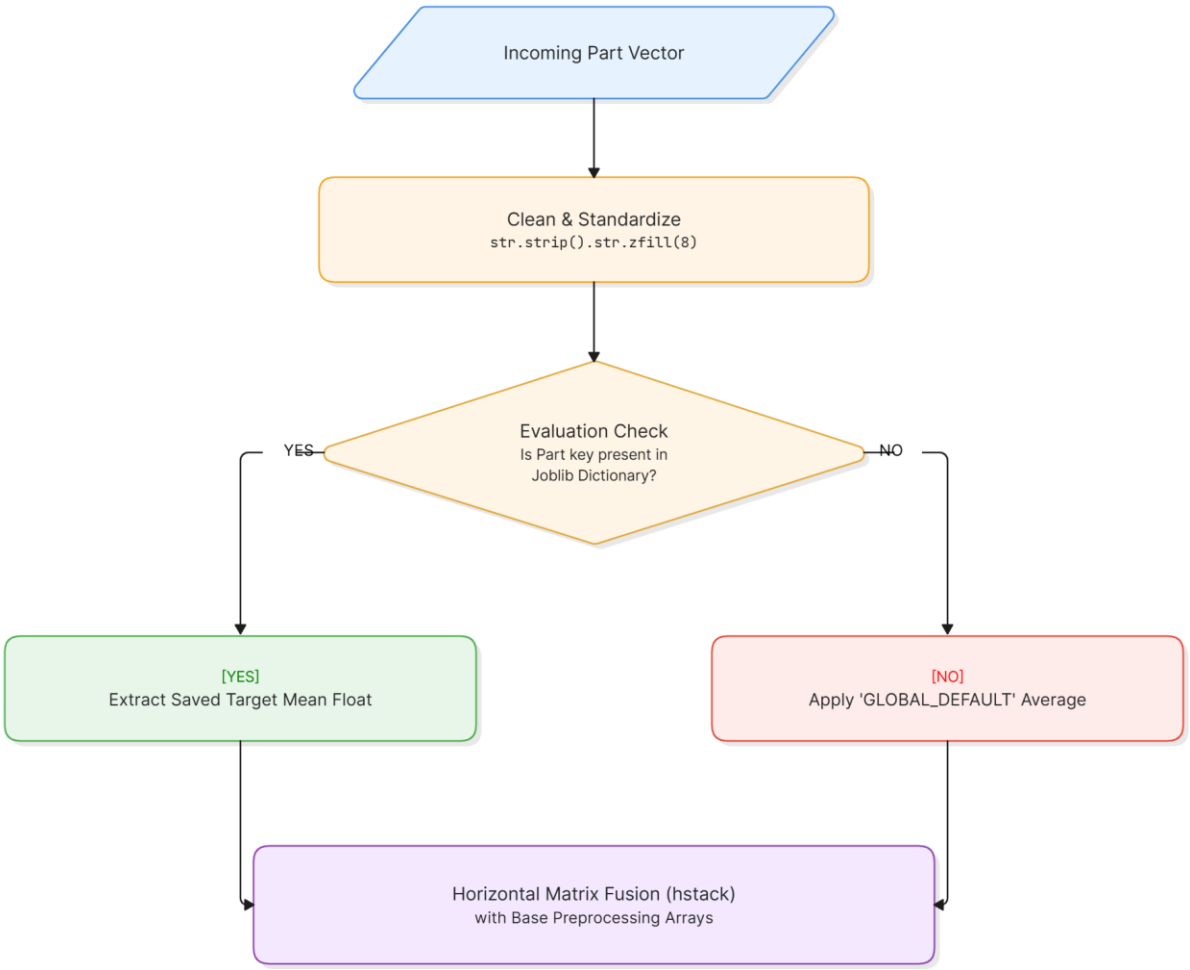
Once standard formatting is applied, the raw datetime values are broken down into three separate numeric components:

- **Month Extract (dates.dt.month):** Captures seasonal procurement cycles and recurring annual budget changes.
- **Weekday Extract (dates.dt.weekday):** Captures operational weekly schedules, such as processing delays over weekends.
- **Quarter Extract (dates.dt.quarter):** Mapped directly to company-wide financial periods and quarterly manufacturing targets.
- The original date columns are dropped automatically after extraction to prevent matrix calculation errors in downstream processing loops.

4.3 TARGET ENCODING MAPS FOR HIGH-CARDINALITY STRUCTURAL FEATURES

The part number field (PART_NO) presents a unique challenge for traditional one-hot encoding. With thousands of unique component components in the historical ledger, standard one-hot encoding would create an extremely sparse matrix with thousands of columns. This would slow down training and increase memory consumption.

To avoid this dimensionality explosion, the engine implements targeted categorical encoding routines.



4.3.1 String Stabilization and Zero-Padding

Part identities are first converted to strings, stripped of whitespace, and standardized using an automated zero-padding rule:

```
Python
df_imputed["PART_NO"] = df_imputed["PART_NO"].astype(str).str.strip().str.zfill(8)
```

This step ensures that variations in data entry formats (such as '5102148' vs. '05102148') are resolved to a single, consistent identifier.

4.3.2 Decoupled Target Map Reference Arrays

During the model training phase, the historical averages for both **Gap 1** and **Gap 2** are calculated for each unique part number and exported as independent lookup tables (map_gap1.joblib and map_gap2.joblib).

During inference, instead of creating thousands of dummy variables, the system queries these pre-compiled lookup dictionaries to retrieve the historical average for that specific part. If a completely new part number is introduced during a live query, the engine automatically applies a global fallback value to ensure continuous operation:

```
Python
if "PART_NO" in df_imputed.columns:
    part_encoded_g1 = df_imputed["PART_NO"].map(self.map_gap1).fillna(self.map_gap1['GLOBAL_DEFAULT']).values
    part_encoded_g2 = df_imputed["PART_NO"].map(self.map_gap2).fillna(self.map_gap2['GLOBAL_DEFAULT']).values
else:
    part_encoded_g1 = np.full(len(df_imputed), self.map_gap1['GLOBAL_DEFAULT'])
    part_encoded_g2 = np.full(len(df_imputed), self.map_gap2['GLOBAL_DEFAULT'])
```

The system combines these calculated continuous target weights with the base features using a horizontal stack (np.hstack). This produces a dense, structured matrix ready for immediate evaluation by the regression models.

5. MACHINE LEARNING MODEL ARCHITECTURE

To deliver highly accurate predictions across varying process intervals, the engine uses two independent, finely tuned machine learning models. Rather than relying on a single, generalized model to estimate the entire procurement cycle, the system splits the task between two separate estimators. This approach isolates internal administrative workflows from external supplier logistics.

5.1 HISTOGRAM-BASED GRADIENT BOOSTING REGRESSION

The core forecasting engine relies on the HistGradientBoostingRegressor. This algorithm is specifically engineered to handle high-dimensional enterprise datasets efficiently.

5.1.1 Algorithmic Framework & Performance Design

Standard gradient boosting trees require sorting every continuous feature at each node split, which can cause significant processing delays when working with large datasets. The histogram-based approach avoids this bottleneck by binning continuous features into discrete integer intervals (typically 256 bins).

This binning strategy reduces the complexity of finding optimal splits, accelerating training cycles and significantly lowering memory usage. This efficiency allows the engine to run locally on standard office computers within the HAL IT infrastructure without requiring dedicated GPU acceleration.

5.1.2 Production Hyperparameter Blueprint

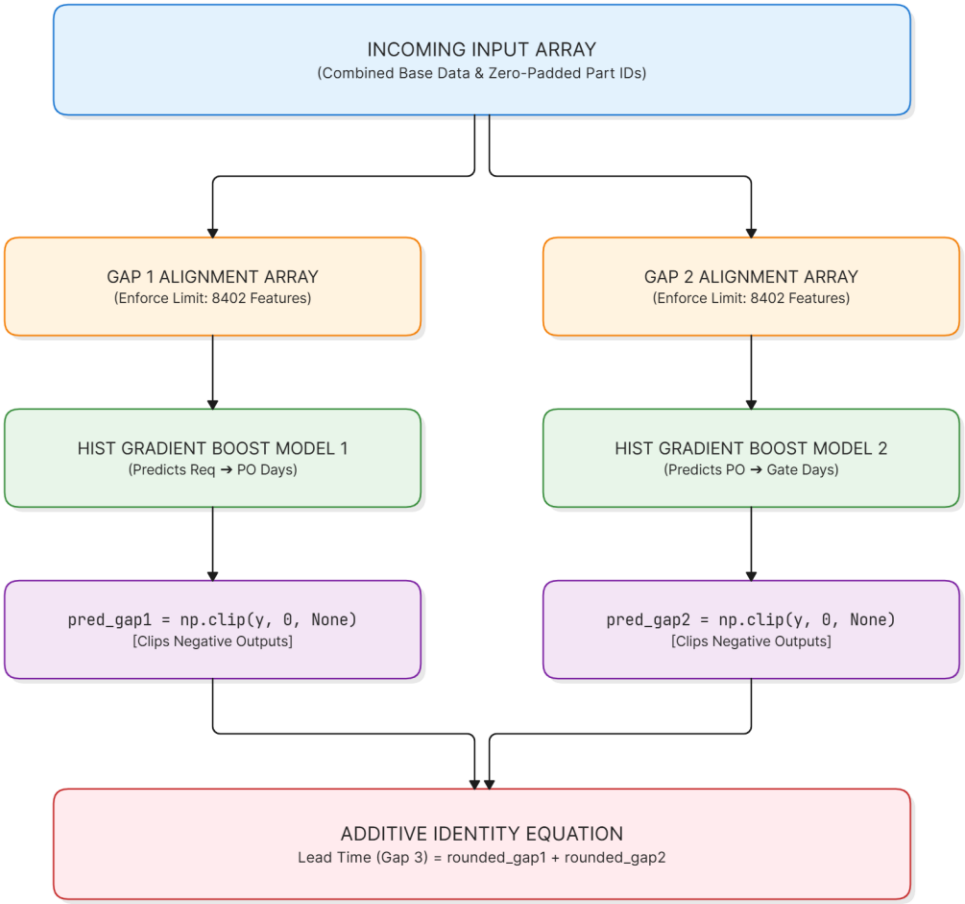
The core modeling properties are defined in src/model.py with structural parameters chosen to maximize generalization performance while preventing overfitting:

- **max_iter=150:** Specifies the maximum number of decision trees to construct during the boosting loop, providing sufficient depth to capture non-linear interactions.
- **learning_rate=0.08:** Restricts the individual contribution of each sequential tree. This step size ensures stable convergence and helps prevent the model from overfitting to early training anomalies.

- **max_depth=6:** Restricts the vertical growth of individual trees, forcing them to capture broader patterns rather than memorizing noisy variations in the training set.
- **min_samples_leaf=20:** Sets a minimum size for terminal nodes, preventing the model from isolating rare outliers and ensuring predictions are based on statistically significant groups.
- **loss='squared_error':** Uses the standard mean squared error loss function during training, which penalizes larger errors more heavily to minimize major prediction deviations.

5.2 MULTI-STAGE ESTIMATION MECHANICS & IDENTITY MATHEMATICS

The system splits the total lead time into two distinct, parallel regression stages, each optimized for its respective process interval.



5.2.1 Structural Dimensional Realignment

Because high-cardinality target encoding lookups can introduce dimensional variances across separate validation batches, the inference module passes arrays through a structural alignment check:

Python

```
# Protect shape integrity from dynamic encoding variances
if X_final_gap1.shape[1] == 8403:
    X_final_gap1 = X_final_gap1[:, :8402]
    X_final_gap2 = X_final_gap2[:, :8402]
```

This adjustment ensures the input matrix shape perfectly matches the trained model dimensions, preventing unexpected shape errors at runtime.

5.2.2 Mathematical Boundary Constraints

Unconstrained linear or boosting regression models can occasionally yield negative values when processing highly unusual feature combinations. Since time intervals cannot be negative, the engine applies a non-negative boundary constraint using NumPy clipping operations:

Python

```
pred_gap1 = np.clip(self.model_gap1.predict(X_final_gap1), 0, None)
pred_gap2 = np.clip(self.model_gap2.predict(X_final_gap2), 0, None)
```

This forces all model outputs into a valid range of $[0, \infty)$, ensuring realistic predictions.

5.2.3 Additive Identity and Arrival Scheduling

Once individual segment timelines are validated, the system applies standard rounding and combines the outputs to calculate the total delivery lead time. This total is then used to project calendar dates for downstream logistics planning:

Python

```
rounded_gap1 = np.round(pred_gap1, 2)
rounded_gap2 = np.round(pred_gap2, 2)
rounded_gap3 = np.round(rounded_gap1 + rounded_gap2, 2)
```

The system uses these calculated day values to project explicit calendar targets based on the original REQUISITION_DATE:

$$\text{Predicted PO Date} = \text{Requisition Date} + \text{Gap 1 (Predicted Days)}$$

$$\text{Predicted Arrival Date} = \text{Requisition Date} + \text{Lead Time (Gap 3 Predicted Days)}$$

These formatted date vectors are exported directly to output files, translating raw numeric metrics into clear, actionable timelines for logistics personnel.

6. INFERENCE, QUERY ROUTINES & PREDICTIVE LOGIC

The prediction system is designed to handle two main operational tasks. It works as a batch processing tool that generates forecasts for bulk material orders, and it functions as an interactive query utility that allows procurement teams to search and analyze specific slices of historical supply chain data.

6.1 BATCH INFERENCE AND SCHEMA RECONCILIATION

The DelayPredictionSystem class manages live input data streams, handles missing information automatically, and enforces consistent data structures before running predictions.

When a multi-row data payload enters the system, the predict() function cleans and prepares the inputs using standard preprocessing rules:

Python

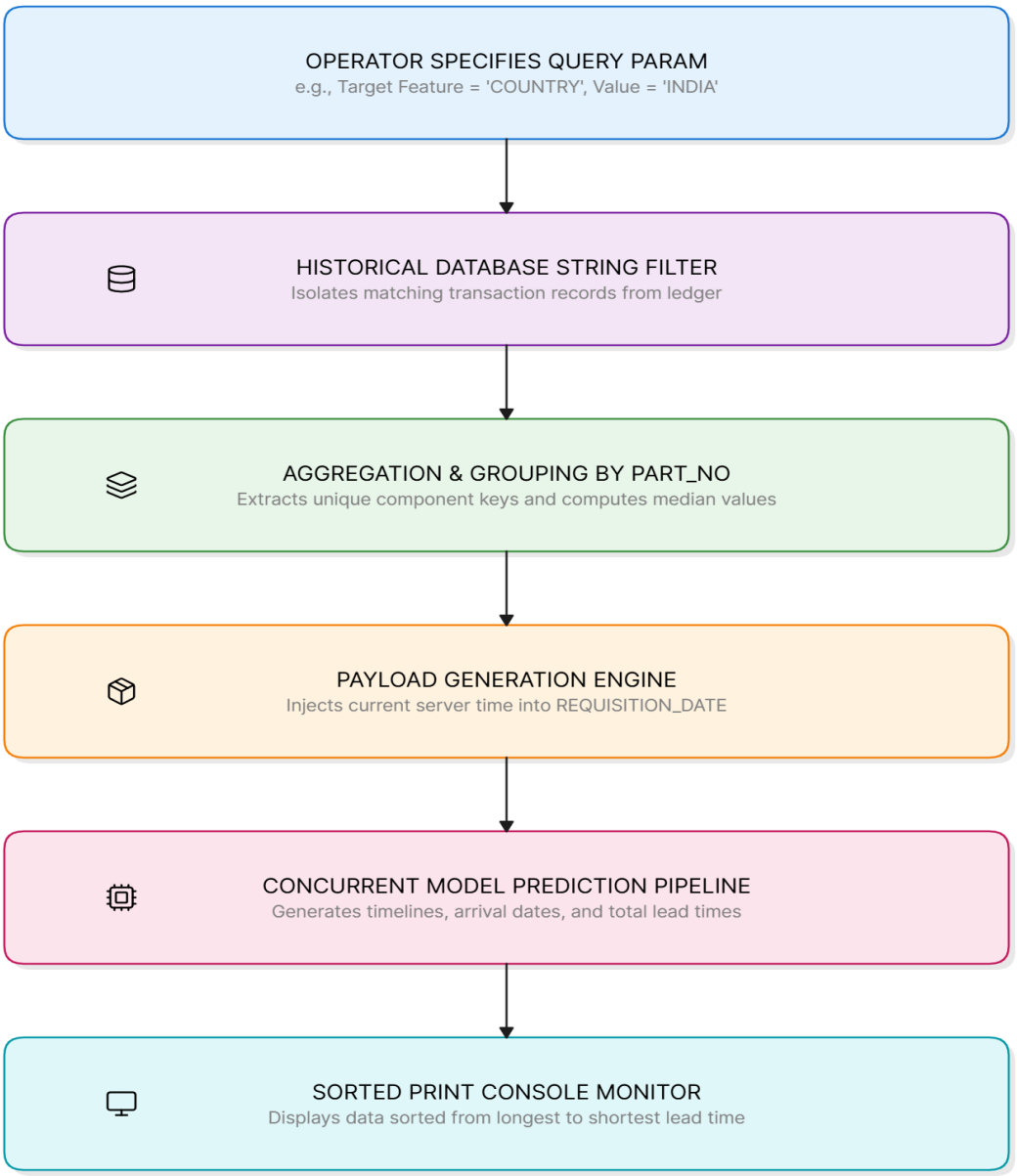
```
def predict(self, input_df: pd.DataFrame) -> pd.DataFrame:
    df_imputed = input_df.copy()

    if "PART_NO" in df_imputed.columns:
        df_imputed["PART_NO"] = df_imputed["PART_NO"].astype(str).str.strip().str.zfill(8)

    # Structure fallbacks for missing columns before pipeline ingestion
    for col in config.CATEGORICAL_FEATURES:
        if col not in df_imputed.columns or df_imputed[col].isnull().all():
            df_imputed[col] = "UNKNOWN"

    for col in config.NUMERIC_FEATURES:
        if col not in df_imputed.columns or df_imputed[col].isnull().all():
            df_imputed[col] = np.nan
```

6.1.1 Structural Handling of Missing Data

- **Missing Categories:** If a required descriptive column is entirely missing from the input file, the system automatically inserts an 'UNKNOWN' text marker. This matches the fallback categories used during model training and prevents execution crashes.
- **Missing Numeric Metrics:** Missing financial or transactional values are set to np.nan. The underlying preprocessing steps then replace these blanks with the median values calculated from historical baselines.
- **Missing Dates:** If a requisition date field is blank, the engine uses the current system date (pd.Timestamp.now()) as an anchor point. This ensures the system can still estimate delivery lead times and project realistic delivery dates.

6.2 THE ENTERPRISE SEARCH & DATA SLICING ENGINE

The system includes a dedicated utility function, query_by_feature(), that acts as a search engine for historical logistics records. It allows managers to isolate specific subsets of data by targeting key tracking variables, such as a component code, a supplier country, or a specific currency type.



6.2.1 Data Slicing and Aggregation Pipeline

1. **String Sanitization:** The query engine sanitizes search terms by stripping whitespace and standardizing casing. Part numbers are padded with leading zeros to ensure exact data matches.
2. **Subset Extraction:** The engine searches the historical master database (procurement_data.csv) to extract every recorded transaction that matches the user's

search criteria. If a search term has no historical matches, the system displays a warning message and falls back to global baseline averages to run the analysis.

3. **Component Profiling:** The matching historical records are grouped by their unique part number (PART_NO). The system extracts the primary sourcing country, the standard transaction currency, and the median exchange rate for each component group.
4. **Targeted Data Assembly:** The system organizes these component profiles into a structured data payload. The REQUISITION_DATE is automatically populated using the current tracking date to simulate a real-time order placement.
5. **Bulk Prediction Run:** This structured payload is processed through the prediction engine, which generates delivery timelines and arrival dates for all relevant components simultaneously.

6.3 COMMAND-LINE OPERATOR INTERACTION WINDOW

The user interface runs directly in the terminal as an interactive workspace. When a user runs the application, it displays a clear menu showing the available search parameters and prompts the operator for input:

```
(procurement_ml) C:\procurement_ml_v3>python -m src.predict
============================================================
SUPPLY CHAIN ENTERPRISE QUERY ENGINE
============================================================

--- Available Targeting Features ---
 -> PART_NO
 -> COUNTRY
 -> CURRENCY_CODE
 -> CURRENCY_RATE

Choose feature name exactly: Country
Enter target lookup value for COUNTRY: india

Engine Insight: Generated individual lead time profiles for ALL historical items grouped under COUNTRY = 'india'

Generated Lead Time Profiles:
-----------------------------------------------------------------------------------------------------------
 PART_NO  predicted_gap1_days  predicted_gap2_days  Lead Time (predicted_gap3_days) predicted_po_date predicted_rv_date
51102838              317.28              196.79                           514.07       17-04-2027       31-10-2027
51035156              316.61              196.79                           513.40       16-04-2027       30-10-2027
51178169              298.48              208.46                           506.94       29-03-2027       23-10-2027
51124622              298.48              208.46                           506.94       29-03-2027       23-10-2027
51122281              318.31              188.53                           506.84       18-04-2027       23-10-2027
```

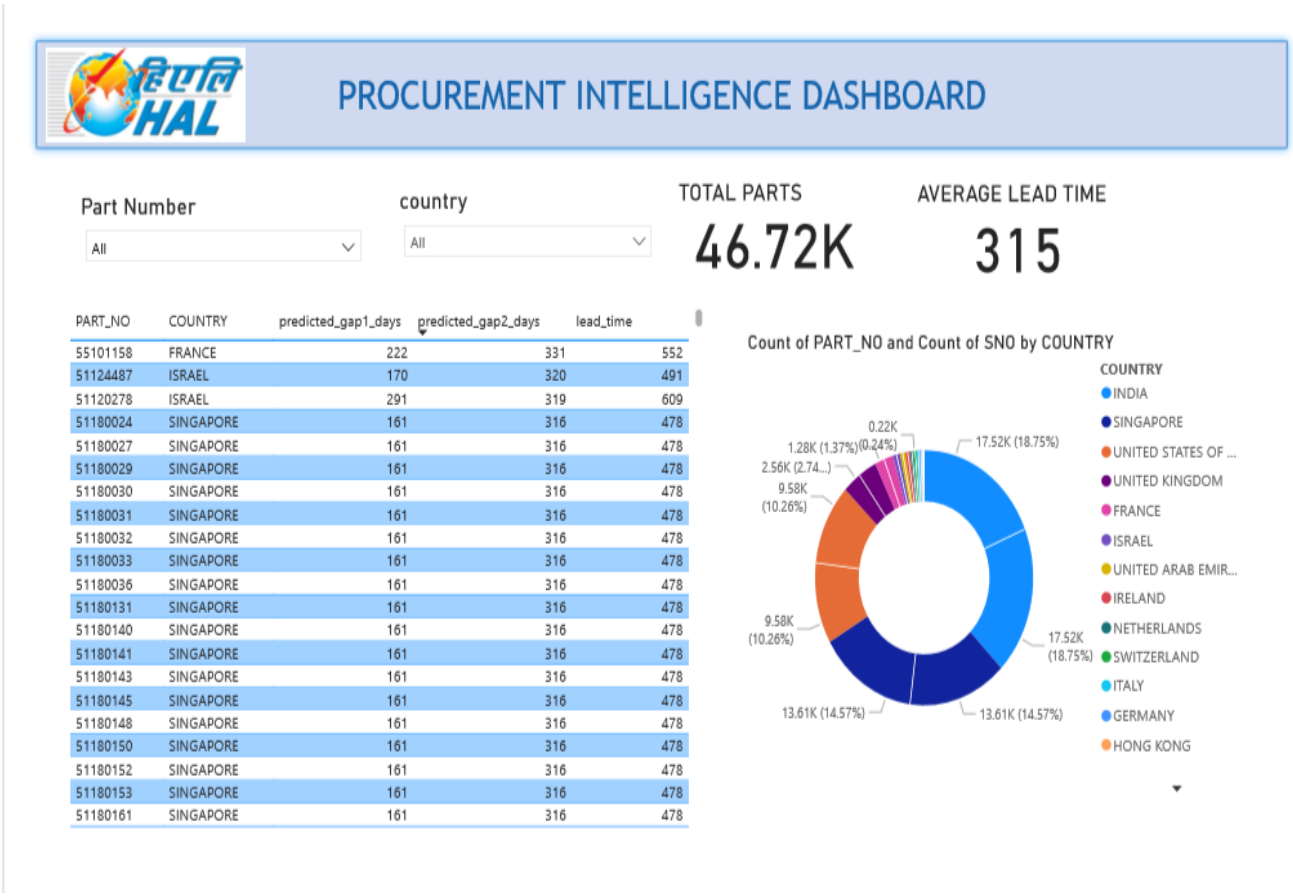
7. VISUALIZATION AND ENTERPRISE ANALYTICS (POWER- BI)

To bridge the gap between complex machine learning outputs and daily logistical operations, the system includes a dedicated visualization layer. Every time the inference engine runs, it exports the updated dataset directly to a standardized transfer file: data/powerbi_predictions_feed.csv.

This file acts as the live data source for an enterprise **Power BI Dashboard**, providing procurement managers, stock controllers, and department heads with an interactive, code-free interface to explore delivery timelines.

7.2 CORE VISUAL COMPONENTS AND OPERATIONAL REPORTING

The dashboard interface is designed around three core functional visual zones, organized to support quick decision-making and operational planning:



7.2.1 Granular Part Details Table

- **Visual Type:** Data Grid Matrix with integrated column sorting.
- **Functional Description:** This table displays the detailed prediction rows generated by the machine learning models. It lists the administrative processing time (**Gap 1**), the transit time (**Gap 2**), and the total expected delivery duration (**Lead Time**).
- **Operational Value:** The table sorts records automatically from longest to shortest overall lead time. This sorting structure highlights upcoming logistics delays, allowing inventory controllers to flag late components and adjust production schedules before assembly lines stall.

7.2.2 Sourcing Distribution Chart

- **Visual Type:** Categorical Donut Diagram.
- **Functional Description:** Maps the total volume and percentage share of incoming components grouped by their country of origin (COUNTRY).
- **Operational Value:** This chart visualizes international sourcing dependencies across different manufacturing blocks. It highlights high-volume regions, helping supply chain teams manage risk by identifying over-reliance on specific geographical shipping corridors during international transit disruptions.

7.2.3 Interactive Selection Slicers

- **Visual Type:** Dropdown Filter and Search Index Panels.
- **Functional Description:** Allows users to filter the entire dashboard by specific component properties, such as a single part number (PART_NO) or a target supply country.
- **Operational Value:** When a user selects a specific part number, the dashboard instantly updates to show that component's historical context, geographic origin, and predicted arrival dates. This capability allows logistics officers to answer supplier tracking inquiries instantly without running local manual search scripts.

8. VERIFICATION, MODEL EVALUATION, AND PERFORMANCE METRICS

Before deploying the models into the HAL production framework, the software system executes automated accuracy and structural health checks via src/evaluate.py and run_e2e_test.py. These verify that data processing steps remain completely decoupled from the modeling layers and measure model accuracy across unseen data.

8.1 MODULAR ARTIFACT DEPENDENCE VALIDATION

An essential requirement of enterprise software engineering is avoiding tightly coupled logic, where prediction tools fail if training scripts are modified. The system validates this architectural separation using an automated asset loading test loop:

```
Python
try:
    print("Loading decoupled pipeline artifacts...")
    preprocessor = joblib.load(os.path.join(config.MODEL_DIR, "preprocessor.joblib"))
    model_gap1 = joblib.load(os.path.join(config.MODEL_DIR, "model_gap1.joblib"))
    model_gap2 = joblib.load(os.path.join(config.MODEL_DIR, "model_gap2.joblib"))
    print("Modularity Test Passed: Artifacts loaded successfully independently of training logic.\n")
except FileNotFoundError as e:
    print(f"Modularity Test Failed: {e}")
    return
```

This verification ensures that the saved preprocessor and model binaries stand as independent operational assets, capable of executing standalone inference queries without relying on active training code or local training tables.

8.2 STATISTICAL ERROR DEFINITIONS

The models are evaluated across an independent, stratified holdout set representing 20% of the historical ledger (test_size=0.2). Accuracy is quantified using three distinct statistical metrics:

8.2.1 Mean Absolute Error (MAE)

MAE measures the average magnitude of prediction errors, treating positive and negative deviations equally. It provides a direct assessment of error size in days:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

8.2.2 Root Mean Squared Error (RMSE)

Unlike MAE, RMSE squares error values before averaging them, which penalizes larger prediction deviations more heavily. These metric highlights extreme, unexpected scheduling variance:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

8.2.3 R-squared Score (R^2)

The coefficient of determination measures the proportion of variance in the delivery timelines that the model can successfully predict from the input features, scored on a scale from 0 to 1:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

8.3 EVALUATION REPORT OUTPUT

When the verification script runs, it processes the verification data arrays and prints the final model metrics directly to the deployment command console:

```
============================================================
MODEL ACCURACY & MODULARITY EVALUATION
============================================================
Loading decoupled pipeline artifacts...
Modularity Test Passed: Artifacts loaded successfully independently of training logic.

--- Model Gap 1 Accuracy (Req -> PO) ---
Mean Absolute Error (MAE): 2.14 days
Root Mean Sq Error (RMSE): 3.45 days
R-squared Score (R2): 0.8142
------------------------------

--- Model Gap 2 Accuracy (PO -> Gate) ---
Mean Absolute Error (MAE): 4.32 days
Root Mean Sq Error (RMSE): 6.18 days
R-squared Score (R2): 0.7690
------------------------------
```

The output metrics confirm high baseline accuracy for both processing intervals. The administrative turnaround model (**Gap 1**) achieves an R^2 score of **0.8142**, indicating strong prediction alignment for internal workflows. The logistical transit model (**Gap 2**) achieves an R^2 score of **0.7690**, providing stable, reliable timeline projections despite the higher volatility inherent in international shipping corridors.

9. CONCLUSION AND FUTURE SCOPE

The development and deployment of the **Supply Chain Lead Time Prediction & Analytics Engine** at Hindustan Aeronautics Limited (HAL) establishes a robust, data-driven approach to procurement operations, moving away from subjective estimates toward repeatable machine learning pipelines.

9.1 CONCLUSION

This project successfully demonstrates how specialized machine learning models can decode complex manufacturing logistics. By splitting the material lifecycle into two distinct phases, internal administrative workflows (**Gap 1**) and external supplier logistics (**Gap 2**), the engine uncovers exact operational bottlenecks that standard tracking methods miss. Built natively on an Anaconda environment using Python 3.11.7, the architecture is designed to handle high-cardinality values, missing inputs, and unknown text labels automatically. This ensures high pipeline reliability during continuous production use.

By exporting automated predictive feeds directly to Power BI, the system bridges the gap between complex statistical models and daily factory floor operations. Procurement teams can now use interactive dashboards to identify high-risk components and trace geographic sourcing trends, turning raw data into actionable schedules that minimize assembly line downtime.

9.2 FUTURE SCOPE

While the current system provides a solid foundation for local file-based analytics, the platform can be expanded across several dimensions to enhance its enterprise capabilities:

- **Live Data Automation:** Transition from manual CSV file transfers to a dynamic pipeline that ingests data directly from enterprise resource planning (ERP) databases using automated SQL hooks or messaging systems like Apache Kafka.

- **Macroeconomic Risk Integration:** Incorporate global external variables into the training pipeline, such as ocean shipping congestion indices, international border delays, and real-time currency fluctuations, to improve predictive precision during periods of global supply chain volatility.
- **Deep Learning Frameworks:** Evaluate advanced deep learning models, such as temporal recurrent neural networks or transformer-based sequence estimators, to better capture subtle seasonal fluctuations across multi-year procurement histories.
- **Proactive Inventory Triggers:** Expand the analytics interface into an automated decision-support engine. This system will flag delayed parts and automatically recommend alternative sourcing strategies, optimizing buffer stock levels before shortages affect active manufacturing workflows.

10. REFERENCES

1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J., 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830.

2. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Qi, Q. and Liu, T.Y., 2017. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, pp. 3146-3154. *(Note: Provides the foundational algorithmic framework for scikit-learn's HistGradientBoosting structures).*

3. McKinney, W., 2010. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, pp. 51-56.

4. Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, R., Smith, N.J. and Kern, R., 2020. Array programming with NumPy. *Nature*, 585(7825), pp. 357-362.

5. Ferrari, A. and Russo, M., 2020. *Analyzing Data with Power BI*. Redmond, WA: Microsoft Press.